

GIT for Beginners

Anthony Baire

Université de Rennes 1 / UMR IRISA

March 21, 2024



This tutorial is licensed under a [Creative Commons Attribution-NonCommercial-NoDerivs 3.0 France License](https://creativecommons.org/licenses/by-nc-nd/3.0/fr/)

Objectives

- Understand the basics about version control systems
- Getting started with GIT
 - working with a local repository
 - synchronising with remote repositories

Summary

1. About Version Control Tools
2. Overview of GIT
3. Working locally
4. Branching & merging
5. Interacting with a remote repository
6. Hosting your code
7. Working with third-party contributors
8. Extras

Part 1.

About Version Control Tools

- Definition
- Use cases
- Base concepts
- History

What is a version control system ?

From: http://en.wikipedia.org/wiki/Revision_control

Revision control [...] is the management of changes to documents, computer programs, large web sites, and other collections of information.

Changes are usually identified by a number or letter code, termed the "revision number" [...]. For example, an initial set of files is "revision 1". When the first change is made, the resulting set is "revision 2", and so on.

Each revision is associated with a timestamp and the person making the change.

Revisions can be compared, restored, and with some types of files, merged.

Use case 2: working with others

VC tools help you to:

- share a collection of files with your team
- merge changes done by other users
- ensure that nothing is accidentally overwritten
- ~~know who you must blame when something is broken~~

Use case 3: branching

You may have multiple variants of the same software, materialised as **branches**, for example:

- a main branch
- a maintenance branch (*to provide bugfixes in older releases*)
- a development branch (*to make disruptive changes*)
- a release branch (*to freeze code before a new release*)

VC tools will help you to:

- handle multiple branches concurrently
- merge changes from a branch into another one

Use case 4: working with external contributors

VC tools help working with third-party contributors:

- it gives them visibility of what is happening in the project
- it helps them to submit changes (patches) and it helps you to integrate these patches
- forking the development of a software and merging it back into mainline³

³decentralised tools only

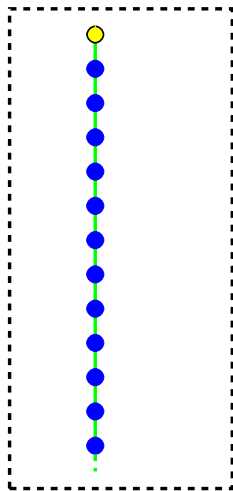
Use case 5: scaling

Some metrics⁴ about the Linux kernel (developed with GIT):

- about 10000 changesets in each new version (every 2 or 3 months)
- 1000+ unique contributors

⁴source: the Linux Foundation

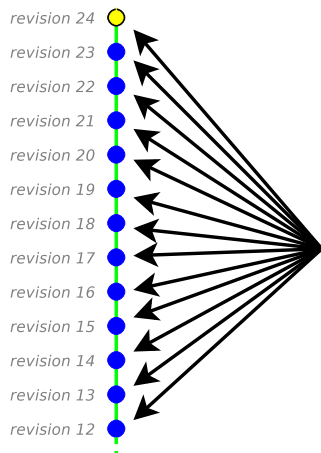
Some illustrations



The Repository

it contains the full history of your project (all revisions from the beginning)

Some illustrations

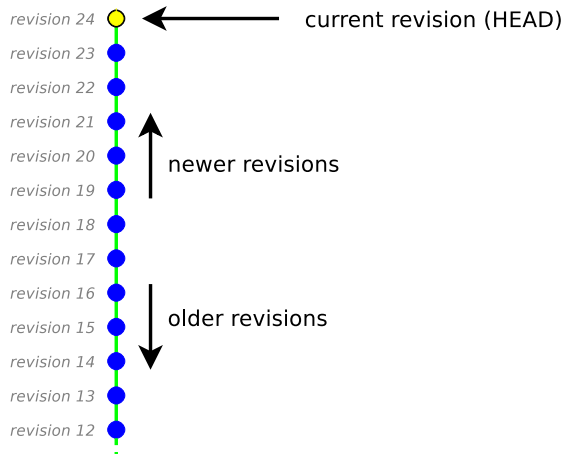


Revisions

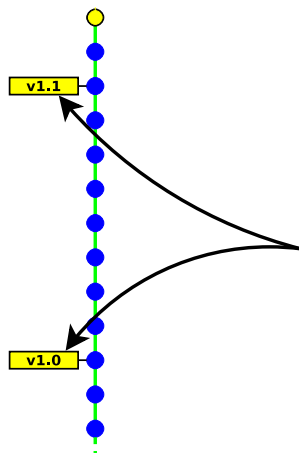
each revision:

- introduces changes from the previous revision
- has an identified author
- contains a textual message describing the changes

Some illustrations



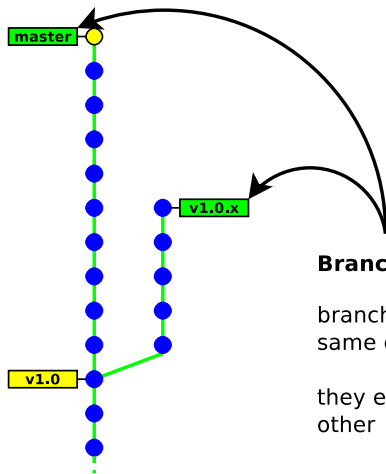
Some illustrations



Tags

a tag identifies a particular revision
(typically each release of the software)

Some illustrations

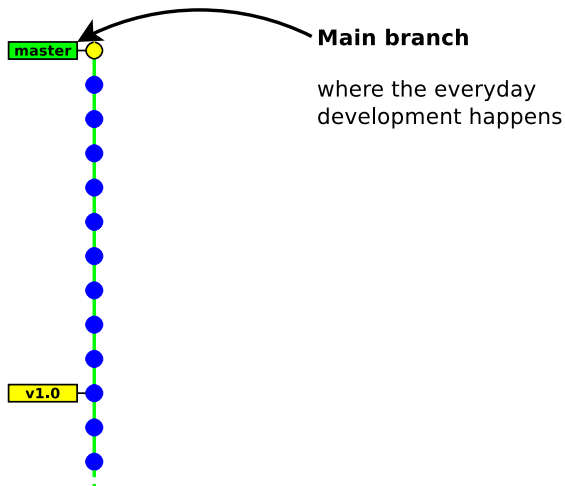


Branches

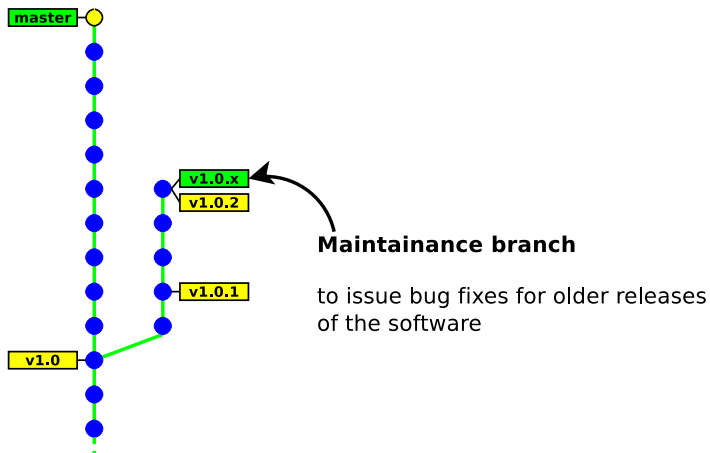
branches are different variants of the same collection of files

they evolve independently of each other

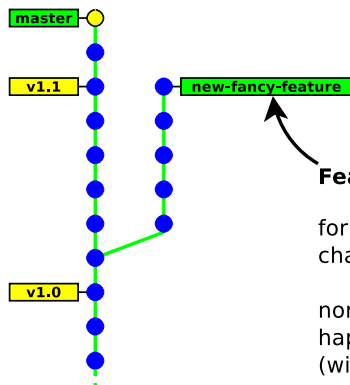
Some illustrations



Some illustrations



Some illustrations

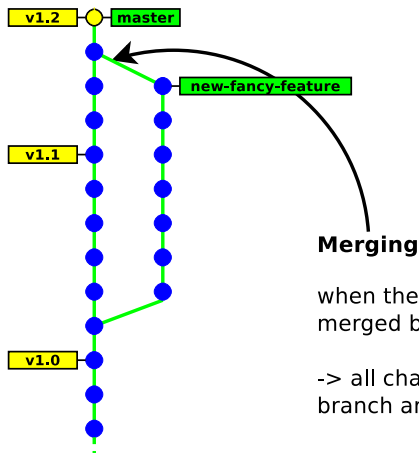


Feature branch

for a new feature requiring intrusive changes in the code

normal development continues to happen in the master branch (without disturbance)

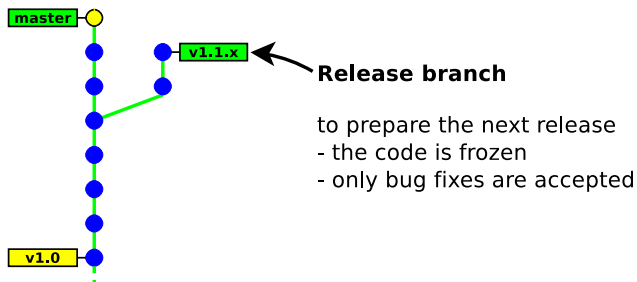
Some illustrations



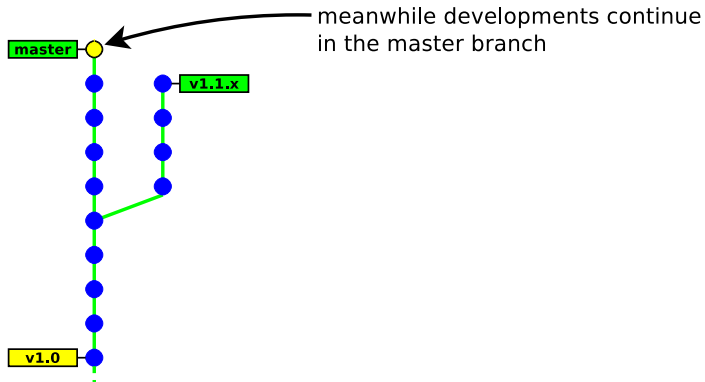
when the new feature is ready, it can be merged back into the master branch

-> all changes done in the feature branch are imported

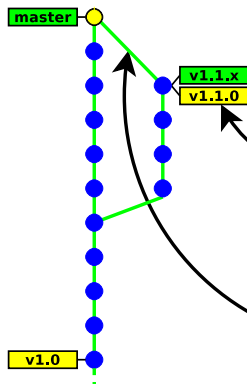
Some illustrations



Some illustrations



Some illustrations

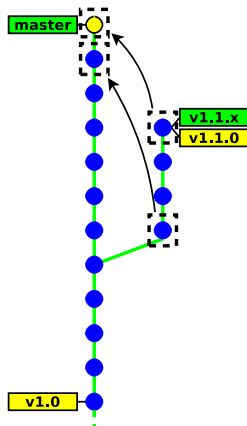


New release

when the code is ready, the new version is released

- the release branch becomes a maintenance branch
- bug fixes can be merged back into the main branch

Some illustrations



Cherry picking

it may not be desirable to merge all the commits into the other branch (e.g. a bug may need a different fix)

-> it is possible to apply each commit individually

Taxinomy

Architecture:

- **centralised** → everyone works on the same unique repository
- **decentralised** → everyone works on his own repository

Concurrency model:

- **lock before edit** (mutual exclusion)
- **merge after edit** (may have conflicts)

History layout:

- **tree** (merges are not recorded)
- **direct acyclic graph**

Atomicity scope: **file** vs **whole tree**

GIT

Other technical aspects

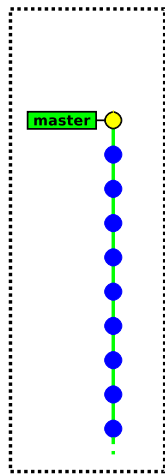
Space efficiency: storing the whole history of a project requires storage space (*storing every revision of every file*)

→ most VC tools use delta compression to optimise the space
(*except Git which uses object packing instead*)

Access method: A repository is identified with a URL. VC tools offer multiple ways of interacting with remote repositories.

- dedicated protocol (*svn:// git://*)
- direct access to a local repository (*file://path* or just *path*)
- direct access over SSH (*ssh:// git+ssh:// svn+ssh://*)
- over http (*http:// https://*)

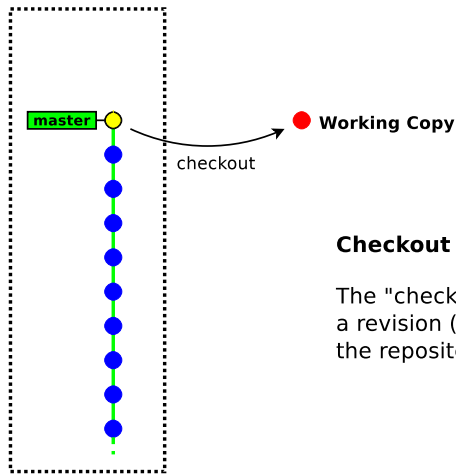
Creating new revisions



A repository is an opaque entity,
it cannot be edited directly

We will first need to extract
a local copy of the files

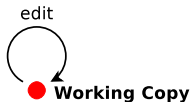
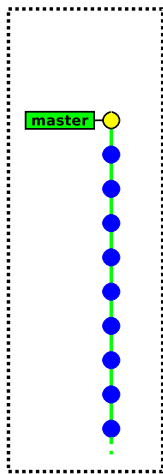
Creating new revisions



Checkout

The "checkout" command extracts a revision (usually the latest) from the repository.

Creating new revisions

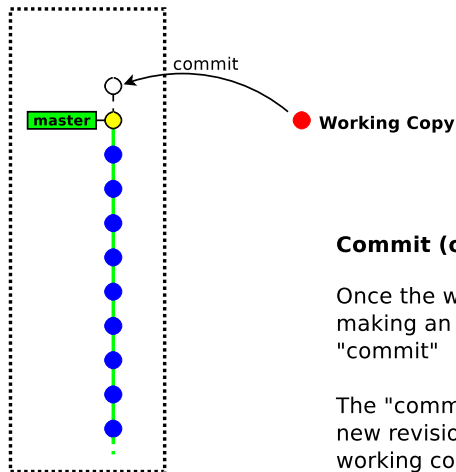


Edition

The working copy is hosted in the local filesystem

It can be edited with any editor,
it can be compiled, ...

Creating new revisions

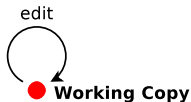
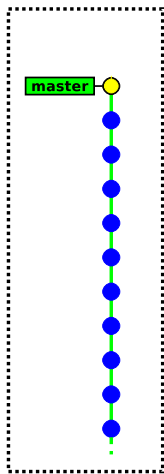


Commit (or Checkin)

Once the working copy is ready for making an new revision, we do a "commit"

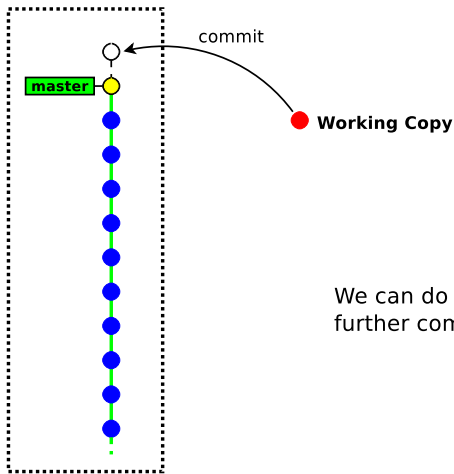
The "commit" command creates a new revision from the current working copy

Creating new revisions



We can do further editions and further commits...

Creating new revisions



We can do further editions and further commits...

What shall be stored into the repository ?

You should store all files that are not generated by a tool:

- source files (`.c .cpp .java .y .l .tex ...`)
- build scripts / project files (`Makefile configure.in Makefile.am CMakefile.txt wscript .sln`)
- documentation files (`.txt README ...`)
- resource files (images, audio, ...)

You should not store generated files

(or you will experience many unnecessary conflicts)

- `.o .a .so .dll .class .jar .exe .dvi .ps .pdf`
- source files / build scripts when generated by a tool (like `autoconf`, `cmake`, `lex`, `yacc`)

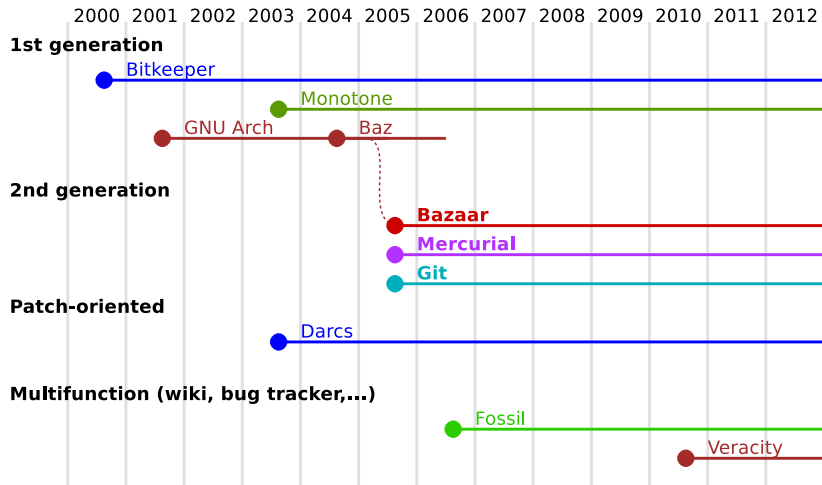
Guidelines for committing

- commit often
- commit independent changes in separate revisions
- in commit messages, describe the rationale behind of your changes (*it is often more important than the change itself*)

History (Centralised Tools)

- 1st generation (*single-file, local-only, lock-before-edit*)
 - 1972: **SCCS**
 - 1982: **RCS**
 - 1985: **PVCS**
- 2nd generation (*multiple-files, client-server, merge-before-commit, contributions through patch files (1985)*)
 - 1986: **CVS**
 - 1992: Rational ClearCase
 - 1994: Visual SourceSafe
- 3rd generation (*+ repository-level atomicity*)
 - 1995: Perforce
 - 2000: **Subversion**
 - + many others

History (Decentralised tools)



Part 2.

Overview of GIT

- History
- Git's design & features
- User interfaces

History

- before 2005: Linux sources were managed with Bitkeeper (proprietary DVCS tool) ⁵
- April 2005: revocation of the free-use licence (because of some reverse engineering)
- No other tools were enough mature to meet Linux's dev constraints (distributed workflow, integrity, performance).
⇒ Linus Torvald started developing Git
- June 2005: first Linux release managed with Git
- December 2005: Git 1.0 released

⁵now open source! (since 2016)

Git Design objectives

- distributed workflow (decentralised)
- easy merging (`merge` deemed more frequent than `commit`)
- integrity (protection against accidental/malicious corruptions)
- speed & scalability
- ease of use

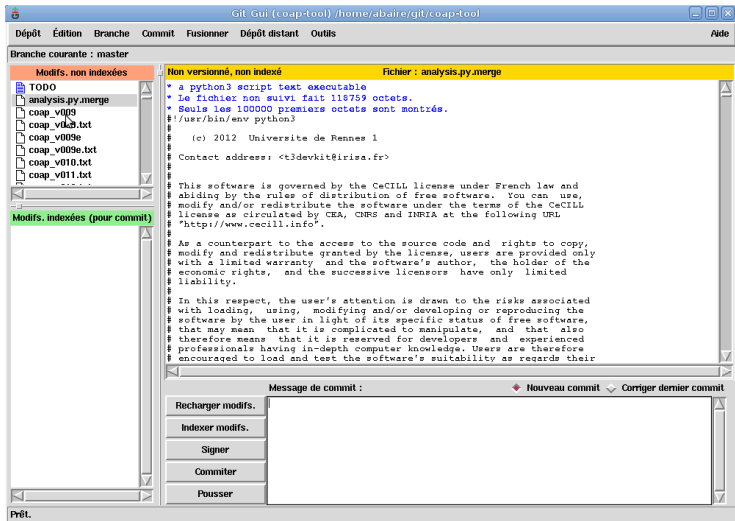
Git Design choices

- Easily hackable
 - simple data structures (blobs, trees, commits, tags)
 - no formal branch history
(a branch is just a pointer to the last commit)
 - low-level commands exposed to the user
- Integrity
 - cryptographic tracking of history (SHA-1 hashes)
 - tag signatures (GPG)
- Merging
 - pluggable merge strategies
 - staging area (index)
- Performance
 - no delta encoding

Git Commands

Version Control Layer	Local commands	add annotate apply archive bisect blame branch check-attr checkout cherry-pick clean commit diff filter-branch grep help init log merge mv notes rebase rerere reset revert rm shortlog show-branch stash status submodule tag whatchanged
	Sync with other repositories	am bundle clone daemon fast-export fast-import fetch format-patch http-backend http-fetch http-push imap-send mailsplit pull push quiltimport remote request-pull send-email shell update-server-info
	Sync with other VCS	archimport cvsexportcommit cvsimport cvsserver svn
	GUI	citool difftool gitk gui instaweb mergetool
VC Low-Level Layer	checkout-index check-ref-format cherry commit-tree describe diff-files diff-index diff-tree fetch-pack fmt-merge-msg for-each-ref fsck gc get-tar-commit-id ls-files ls-remote ls-tree mailinfo merge-base merge-file merge-index merge-one-file mergetool--lib merge-tree mktag mktree name-rev pack-refs parse-remotes patch-id prune read-tree receive-pack reflog replace rev-list rev-parse send-pack show show-ref sh-setup strip-space symbolic-ref update-index update-ref upload-archive verify-tag write-tree	
Utilities	config var web--browse	
Database Layer	cat-file count-objects hash-object index-pack pack-objects pack-redundant prune-packed relink repack show-index unpack-file unpack-objectsupload-pack verify-pack	
Database (blobs, trees, commits, tags)		

Git GUIs: git gui → preparing commits



3rd party GUIs

<https://git-scm.com/downloads/guis>

- Anchorpoint
- Aurees
- CommandGit
- Cong
- Fork
- Fugitive
- Git Extensions
- Git Klient
- GitAhead
- GitAtomic
- GitDrive
- GitFiend
- GitFinder
- GitHub Desktop
- GitKraken
- GitUI
- GitUp
- GitViewer
- GitVine
- Gitfox
- Gitgui
- Gitnuro
- Gittyup
- Glint
- Guitar
- LazyGit
- Magit
- MeGit
- NitroGit
- Pocket Git
- PolyGit
- RepoZ
- SmartGit
- SnailGit
- SourceTree
- Sublime Merge
- TortoiseGit
- Tower
- Vershd
- Working Copy
- giggle
- git-cola
- gitg
- gitionic
- ungit

Part 3.

Working locally

- creating a repository
- adding & committing files
- the staging area (or index)

Create a new repository

```
git init myrepository
```

This command creates the directory *myrepository*.

- the repository is located in *myrepository/.git*
- the (initially empty) working copy is located in *myrepository/*

```
$ pwd
/tmp
$ git init helloworld
Initialized empty Git repository in /tmp/helloworld/.git/
$ ls -a helloworld/
.  ..  .git
$ ls helloworld/.git/
branches  config  description  HEAD  hooks  info  objects  refs
```

Note: The */.git/* directory contains your whole history,
 **do not delete it**⁶

⁶unless your history is merged into another repository

Commit your first files

```
git add file  
git commit [ -m message ]
```

```
$ cd helloworld  
$ echo 'Hello World!' > hello  
$ git add hello  
$ git commit -m "added file 'hello'"  
[master (root-commit) e75df61] added file 'hello'  
1 files changed, 1 insertions(+), 0 deletions(-)  
create mode 100644 hello
```

Note: “master” is the name of the default branch created by
`git init`

The staging area (aka the “index”)

Usual version control systems provide two spaces:

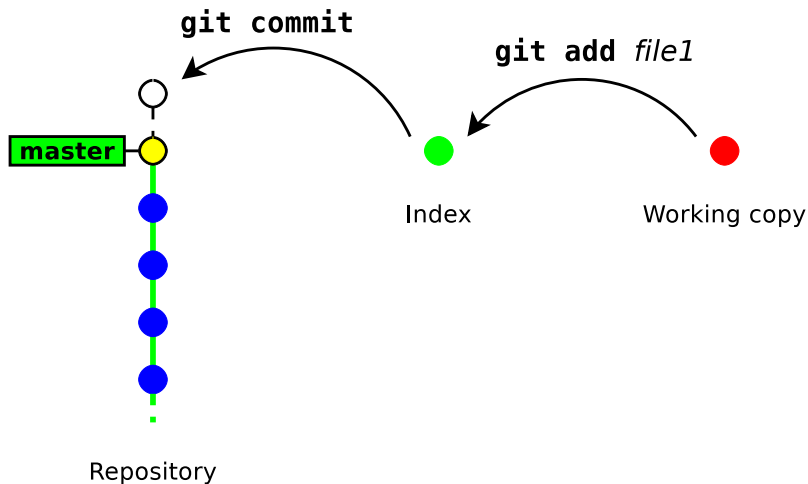
- the **repository**
(the whole history of your project)
- the **working tree** (or **local copy**)
(the files you are editing and that will be in the next commit)

Git introduces an intermediate space : the **staging area**
(also called **index**)

The index stores the files scheduled for the next commit:

- `git add files` → copy files into the index
- `git commit` → commits the content of the index

The staging area (aka the “index”)



Update a file

```
$ echo 'blah blah blah' >> hello
$ git commit
# On branch master
# Changed but not updated:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#       modified:   hello
#
no changes added to commit (use "git add" and/or "git commit -a")
```

Git complains because the index is unchanged (nothing to commit)

→ We need to run **git add** to copy the file into the index

```
$ git add hello
$ git commit -m "some changes"
[master f37f2cf] some changes
1 files changed, 1 insertions(+), 0 deletions(-)
```

Bypassing the index⁷

Running `git add` & `git commit` for every iteration is tedious.

GIT provides a way to bypass the index.

```
git commit file1 [ file2 ...]
```

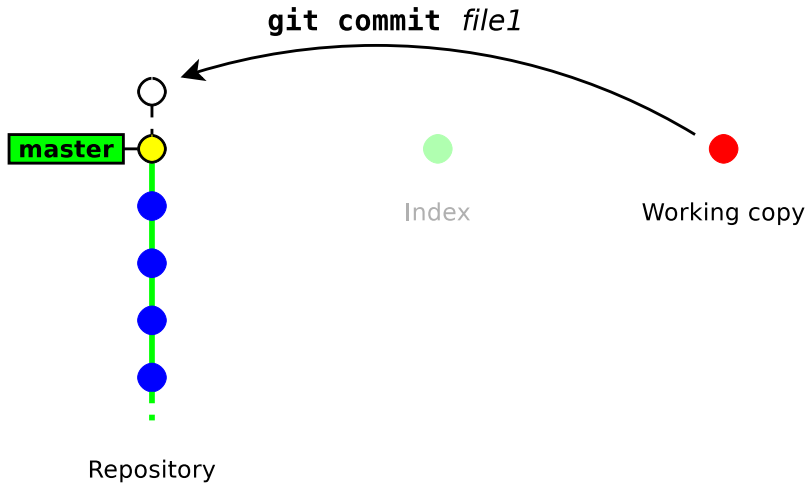
This command commits files (or dirs) directly from the working tree

Note: when bypassing the index, GIT ignores new files:

- “`git commit .`” commits only files that were present in the last commit (updated files)
- “`git add . && git commit`” commits everything in the working tree (including new files)

⁷also named “partial commit”

Bypassing the index



Deleting files

```
git rm file
```

→ remove the file from the index and from the working copy

```
git commit
```

→ commit the index

```
$ git rm hello
rm 'hello'
$ git commit -m "removed hello"
[master 848d8be] removed hello
1 files changed, 0 insertions(+), 3 deletions(-)
delete mode 100644 hello
```

Showing differences

```
git diff [ rev_a [ rev_b ] ] [ -- path ...]
```

→ shows the differences between two revisions *rev_a* and *rev_b*
(*in a format suitable for the patch utility*)

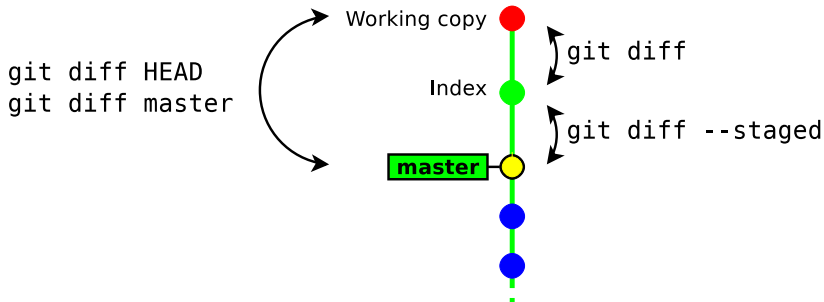
- by default *rev_a* is the **index**
- by default *rev_b* is the **working_copy**

```
git diff --staged [ rev_a ] [ -- path ...]
```

→ shows the differences between *rev_a* and the index

- by default *rev_a* is HEAD (*a symbolic references pointing to the last commit*)

About `git diff` and the index



Diff example

```
$ echo foo >> hello
$ git add hello
$ echo bar >> hello
```

```
$ git diff
--- a/hello
+++ b/hello
@@ -1,2 +1,3 @@
  Hello World!
  foo
+bar
```

```
$ git diff --staged
--- a/hello
+++ b/hello
@@ -1 +1,2 @@
  Hello World!
+foo
```

```
$ git diff HEAD
--- a/hello
+++ b/hello
@@ -1 +1,3 @@
  Hello World!
+foo
+bar
```

Resetting changes

```
git reset [ --hard ] [ -- path ...]
```

`git reset` cancels the changes in the index (and possibly in the working copy)

- `git reset` drops the changes staged into the index⁸, but the working copy is left intact
- `git reset --hard` drops all the changes in the index **and** in the working copy

⁸it restores the files as they were in the last commit

Resetting changes in the working copy⁹

```
git checkout -- path
```

This command restores a file (or directory) as it appears in the index (thus it drops all unstaged changes)

```
$ git diff HEAD
--- a/hello
+++ b/hello
@@ -1 +1,3 @@
   Hello World!
+foo
+bar
$ git checkout -- .
$ git diff HEAD
--- a/hello
+++ b/hello
@@ -1 +1,2 @@
   Hello World!
+foo
```

⁹since v2.23 you may also use the experimental command `git restore`

Other local commands

- `git status` → show the status of the index and working copy
- `git show` → show the details of a commit (metadata + diff)
- `git log` → show the history
- `git mv` → move/rename a file¹⁰
- `git tag` → creating/deleting tags (to identify a particular revision)

¹⁰note that `git mv` is strictly equivalent to: `cp src dst && git rm src && git add dst` (file renaming is not handled formally, but heuristically)

Exercises

1. create a new repository
2. create a new file, add it to the index and commit it
3. launch `gitk` to display it. Keep the window open and hit F5 after each command (to visualise the results of your commands)
4. modify the file and make a new commit
5. rename the file (either with `git mv` or `git add+git rm`), do a `git status` before committing (to ensure the renaming is correctly handled)
6. delete the file and commit it
7. create two new files and commit them. Then modify their content in the working copy and display the changes with `git diff`
8. add one file into the index but keep the other one. Display the changes between:
 - the index and the working copy
 - the last commit and the index
 - the last commit and the working copy
9. run `git reset` to reset the index
10. run `git reset --hard` to reset the index and the working copy

Part 4.

Branching & merging

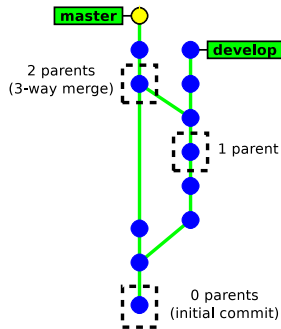
- How GIT handles its history
- Creating new branches
- Merging & resolving conflicts

How GIT handles its history

Each **commit** object has a list of **parent commits**:

- 0 parents → initial commit
- 1 parent → ordinary commit
- 2+ parents → result of a **merge**

→ This is a Direct Acyclic Graph



How GIT handles its history

- There is no formal “branch history”
 - a **branch** is just a pointer on the latest commit.
(git handles branches and tags in the same way internally)
- Commits are identified with **SHA-1 hash** (160 bits) computed from:
 - the committed files
 - the meta data (commit message, author name, ...)
 - the hashes of the parent commits
 - A commit id (hash) identifies **securely** and **reliably** its content and all the previous revisions.

Creating a new branch

```
git checkout -b new_branch [ starting_point ]
```

- *new_branch* is the name of the new branch
- *starting_point* is the starting location of the branch (possibly a commit id, a tag, a branch, ...). If not present, git will use the current location.

```
$ git status
# On branch master
nothing to commit (working directory clean)
$ git checkout -b develop
Switched to a new branch 'develop'
$ git status
# On branch develop
nothing to commit (working directory clean)
```

Switching between branches¹¹

```
git checkout [-m] branch_name
```

```
$ git status
# On branch develop
nothing to commit (working directory clean)
$ git checkout master
Switched to branch 'master'
```

Note: it may fail when the working copy is not clean. Add `-m` to request merging your local changes into the destination branch.

```
$ git checkout master
error: Your local changes to the following files would be overwritten by checkout: hello
Please, commit your changes or stash them before you can switch branches.
Aborting
$ git checkout -m master
M       hello
Switched to branch 'master'
```

¹¹since v2.23 you may also use the experimental command `git switch`

Merging a branch

```
git merge other_branch
```

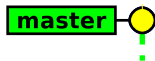
This will merge the changes in *other_branch* into the current branch.

```
$ git status
# On branch master
nothing to commit (working directory clean)
$ git merge develop
Merge made by recursive.
 dev |    1 +
hello |   4 +++-
2 files changed, 4 insertions(+), 1 deletions(-)
create mode 100644 dev
```

Notes about merging

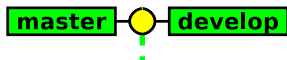
- The result of `git merge` is immediately committed (unless there is a conflict)
- The new commit object has **two parents**.
 - the merge history is recorded
- `git merge` applies only the changes since the last common ancestor in the other branch.
 - if the branch was already merged previously, then only the changes since the last `merge` will be merged.

Branching example



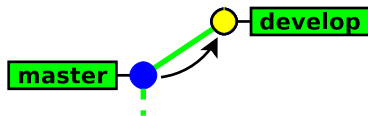
Branching example

```
git checkout -b develop
```



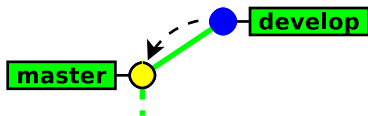
Branching example

`git commit`



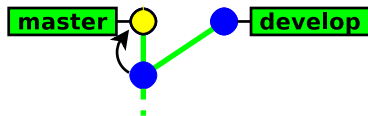
Branching example

`git checkout master`



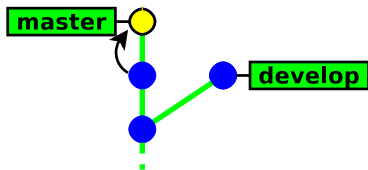
Branching example

`git commit`



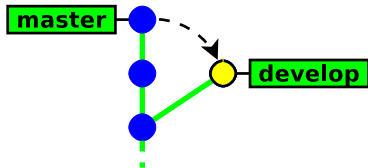
Branching example

`git commit`



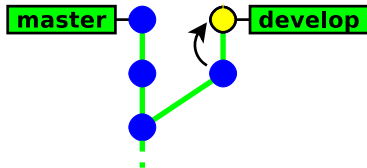
Branching example

`git checkout develop`



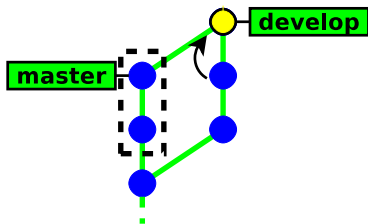
Branching example

`git commit`



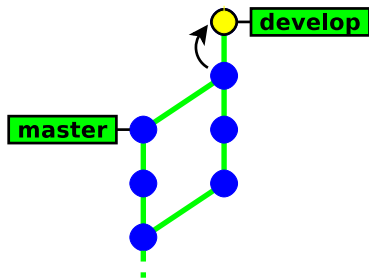
Branching example

`git merge master`



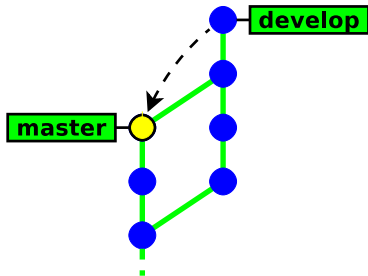
Branching example

`git commit`



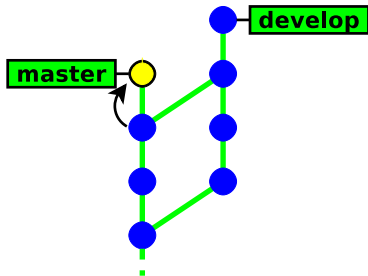
Branching example

`git checkout master`



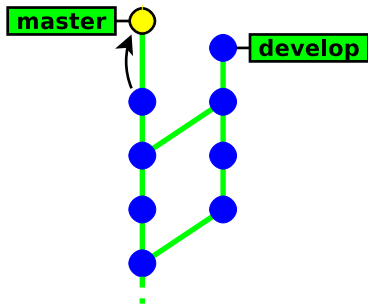
Branching example

`git commit`



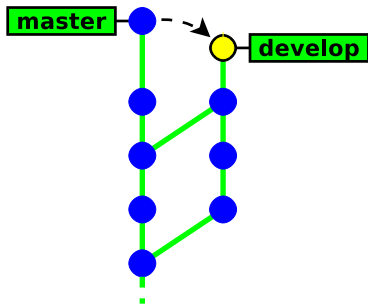
Branching example

`git commit`



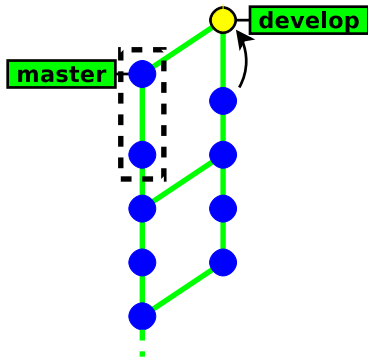
Branching example

`git checkout develop`

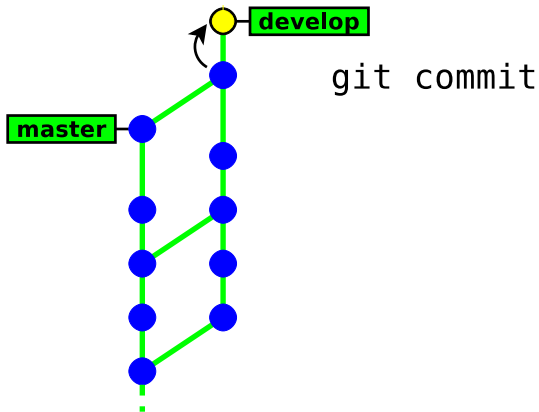


Branching example

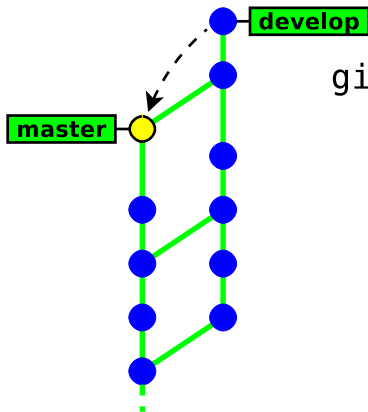
git merge master



Branching example

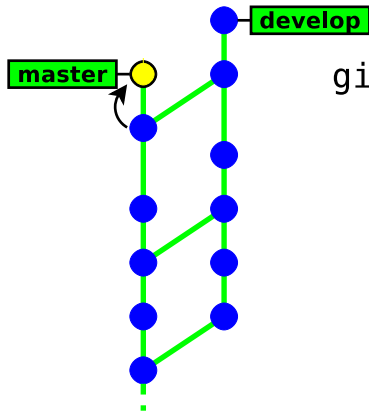


Branching example

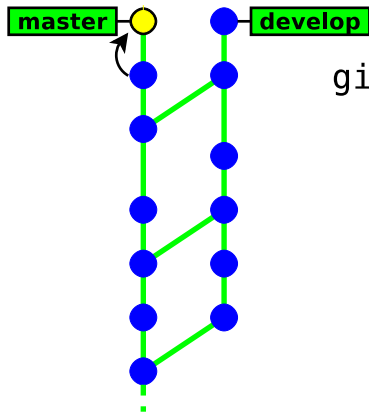


git checkout master

Branching example

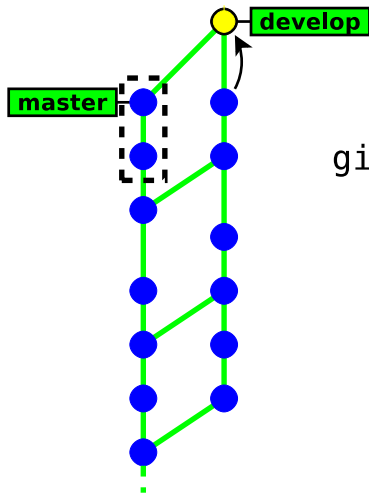


Branching example



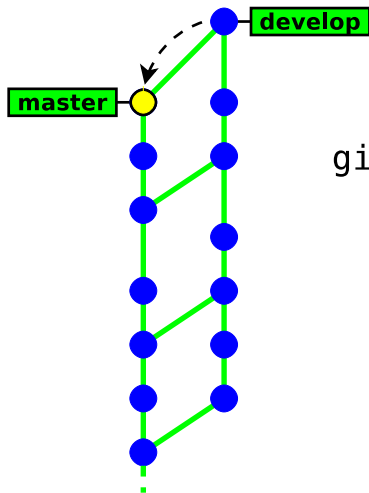
git commit

Branching example



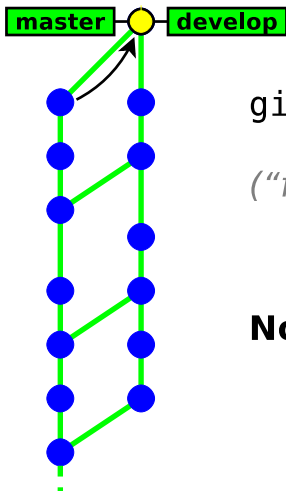
git merge master

Branching example



git checkout master

Branching example



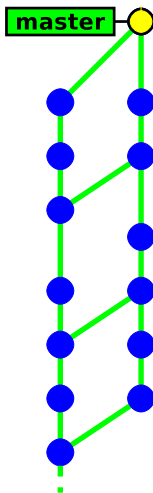
`git merge develop`

("fast-forward" case)

Note:

now the two branches
share **exactly**
the same history

Branching example



```
git branch -d develop
```

How Git merges files ?

If the same file was independently modified in the two branches, then Git needs to merge these two variants

- **textual files** are merged on a per-line basis:
 - lines changed in only one branch are automatically merged
 - if a line was modified in the two branches, then Git reports a conflict. Conflict zones are enclosed within <<<<<< >>>>>>

```
Here are lines that are either unchanged from the common
ancestor, or cleanly resolved because only one side changed.
<<<<<< yours:sample.txt
Conflict resolution is hard;
let's go shopping.
=====
Git makes conflict resolution easy.
>>>>>> theirs:sample.txt
And here is another line that is cleanly resolved or unmodified.
```

- **binary files** always raise a conflict and require manual merging

Merge conflicts

In case of a conflict:

- **unmerged files** (those having conflicts) are left **in the working tree** and marked as “unmerged”¹²
- **the other files** (free of conflicts) and the metadata (commit message, parents commits, ...) are automatically added **into the index** (the staging area)

¹²Git will refuse to commit the new revision until all the conflicts are explicitly resolved by the user

Resolving conflicts

There are two ways to resolve conflicts:

- either edit the files manually, then run

```
git add file    → to check the file into the index
```

or

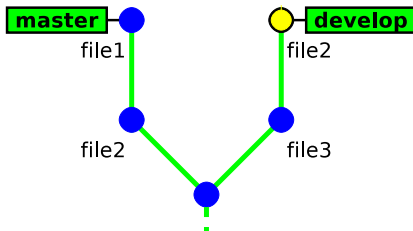
```
git rm file     → to delete the file
```

- or with a conflict resolution tool(xxdiff, kdiff3, emerge, ...)

```
git mergetool [ file ]
```

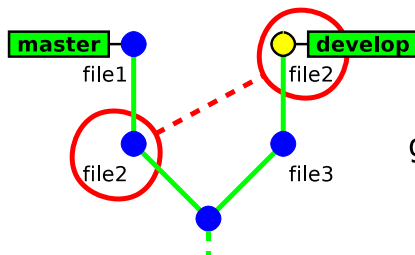
Then, once all conflicting files are checked in the index, you just need to run `git commit` to commit the merge.

Conflict example



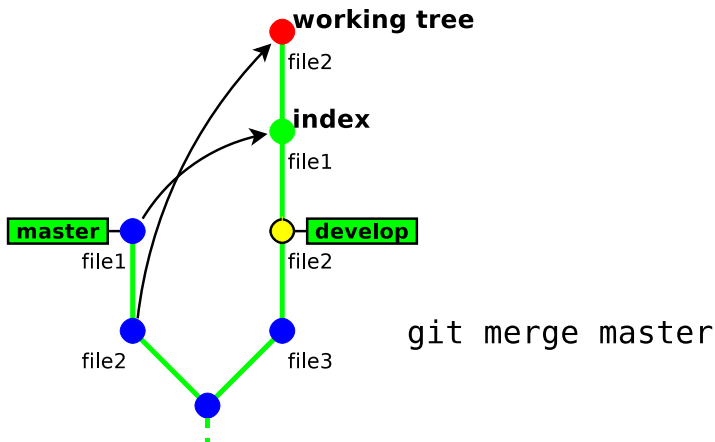
Conflict example

!! conflict !!

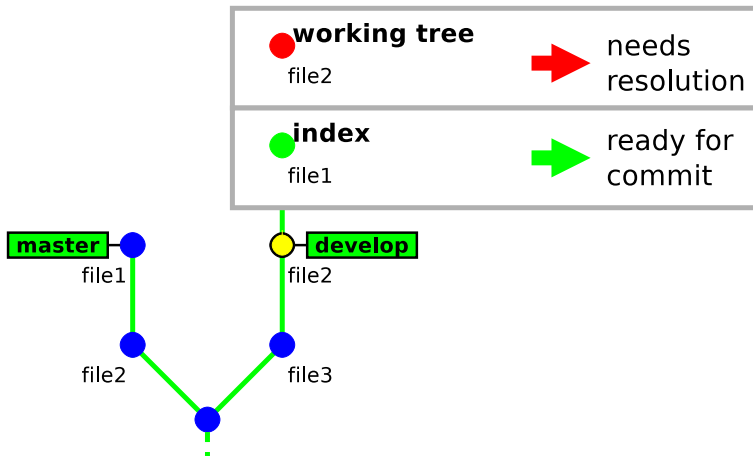


git merge master

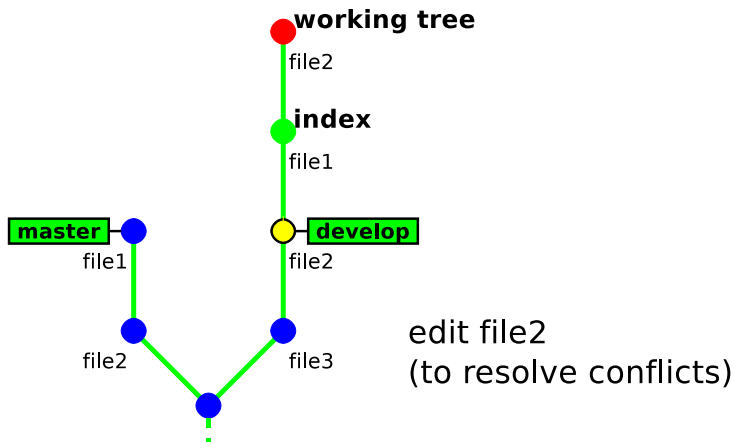
Conflict example



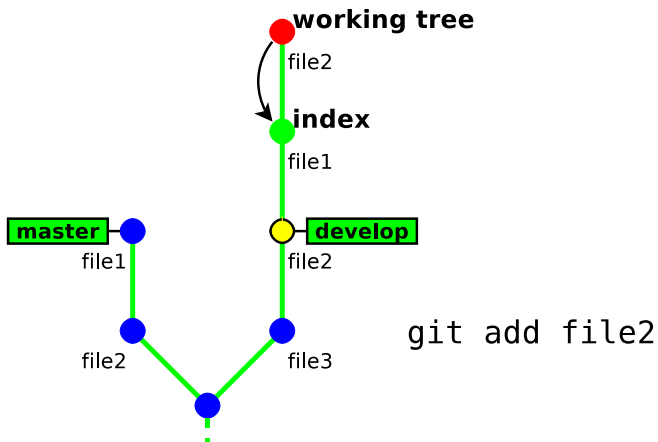
Conflict example



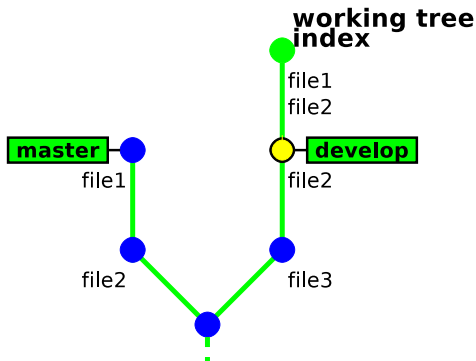
Conflict example



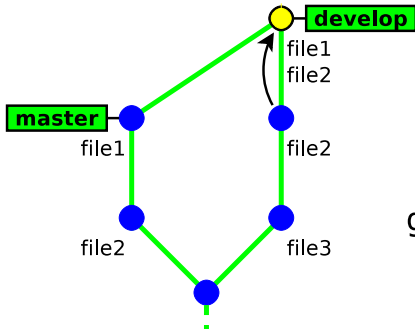
Conflict example



Conflict example



Conflict example



git commit

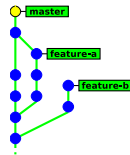
Deleting branches

```
git branch -d branch_name
```

This command has some restrictions, it cannot delete:

- the current branch (HEAD)
- a branch that has not yet been merged into the current branch

```
$ git branch -d feature-a
Deleted branch feature-a (was 45149ea).
$ git branch -d feature-b
error: The branch 'feature-b' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feature-b'.
$ git branch -d master
error: Cannot delete the branch 'master' which you are currently on.
```



→ `git branch -d` is safe¹³

¹³unlike `git branch -D` which **deletes unconditionally** (⚠) the branch

Exercises

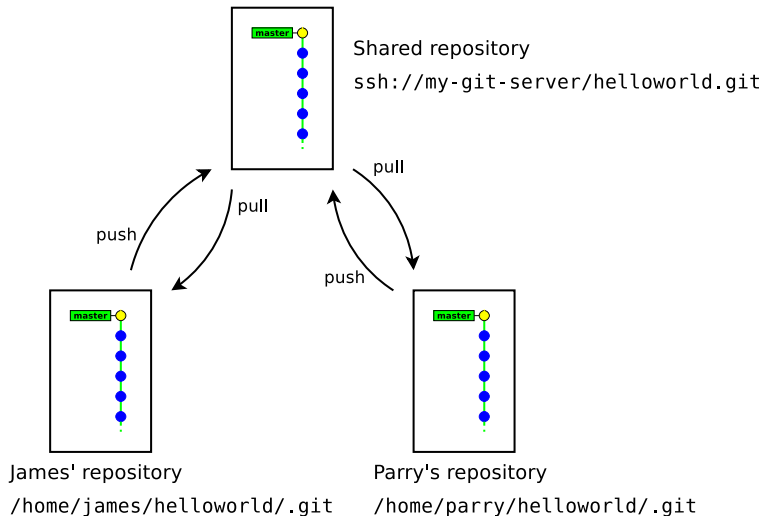
0. use “gitk --all” to display all branches
(and remember to hit F5 after each command to visualise the changes)
1. create a new branch named “develop”
2. make some commits in this branch
3. go back to branch “master” and make some commits
4. merge branch “develop” into “master”
5. make a new commit in each branch so as to generate a conflict (edit the same part of a file)
6. merge branch “develop” into “master”, and fix the conflict
7. merge “master” into “develop”

Part 5.

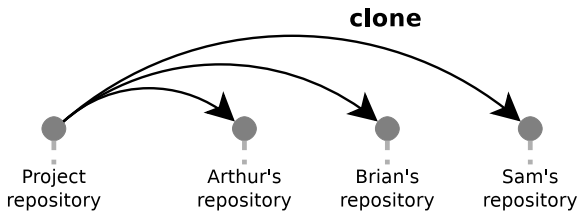
Interacting with a remote repository

- Overview
- Creating a shared repository
- Configuring a remote repository
- Sending changes (push)
- Receiving changes (pull)

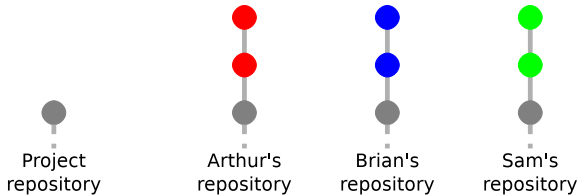
Simple workflow (Centralised)



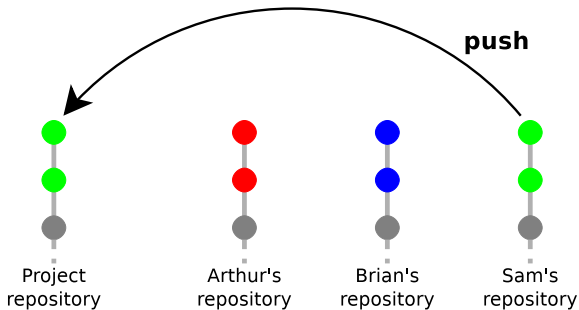
Team Workflow



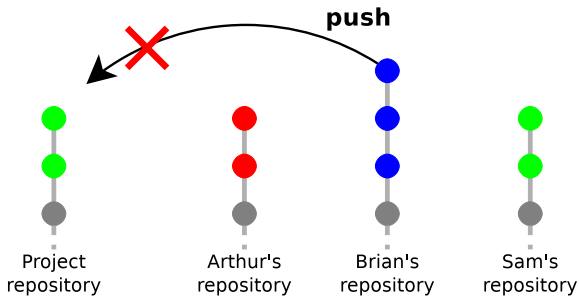
Team Workflow



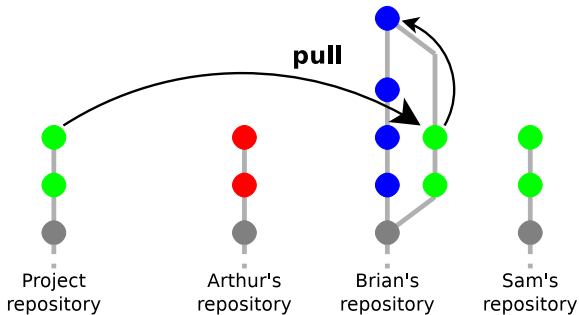
Team Workflow



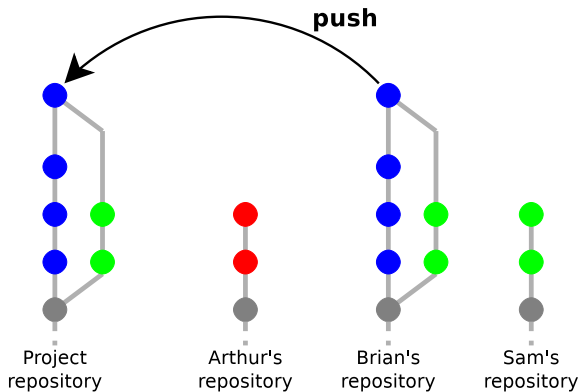
Team Workflow



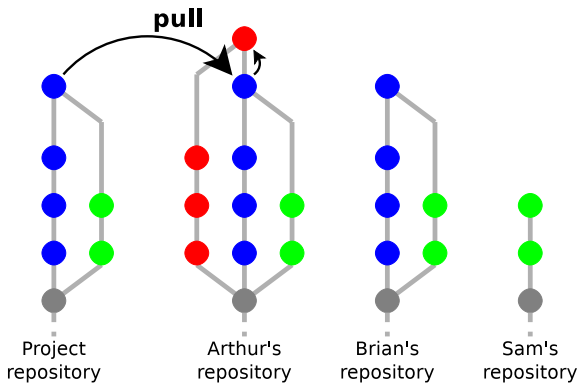
Team Workflow



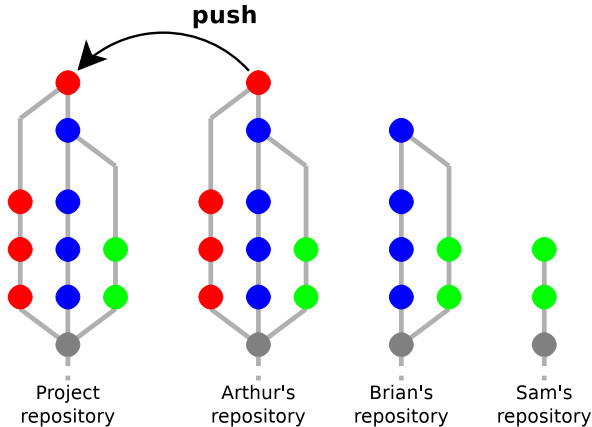
Team Workflow



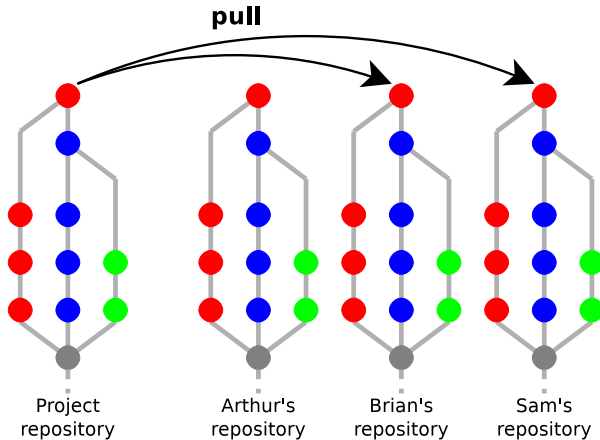
Team Workflow



Team Workflow



Team Workflow



How git handles remote repositories

- It is possible to work with multiple remote repositories
- Each remote repository is identified with a local alias.
When working with a unique remote repository, it is usually named **origin**¹⁴
- Remote repositories are mirrored within the local repository
- Remote branches are mapped in a separate namespace:
`remote/name/branch`.
Examples:
 - `master` refers to the local master branch
 - `remote/origin/master` refers to the master branch of the remote repository named origin

¹⁴default name used by `git clone`

Adding a remote repository

```
git remote add name url
```

- *name* is a local alias identifying the remote repository
- *url* is the location of the remote repository

Examples:

```
$ git remote add origin /tmp/helloworld.git
```

```
$ git remote add origin ssh://username@scm.gforge.inria.fr/gitroot/helloworld/helloworld.git
```

Pushing (uploading) local changes to the remote repository

```
git push
```

- `git push` examines the current branch, then:
 - if the branch is tracking an upstream branch, then the local changes (commits) are propagated to the remote branch
 - if not, then nothing is sent
(*new branches created locally are considered private by default*)
- In case of conflict `git push` will fail and require to run `git pull` first

Pushing a new branch to the remote repository

```
git push -u destination_repository ref [ref ...]
```

- explicit variant of **git push**: the local reference *ref* (a branch or a tag) is pushed to the remote *destination_repository*
- **-u/--set-upstream** configures the local branch to track the remote branch¹⁵ (this is usually what you want)

```
$ git push
```

```
fatal: The current branch master has no upstream branch.
```

```
To push the current branch and set the remote as upstream, use
```

```
git push --set-upstream origin master
```

```
$ git push -u origin master
```

```
To /tmp/helloworld.git/
```

```
* [new branch]      master -> master
```

```
Branch master set up to track remote branch master from origin.
```

¹⁵ so that **git pull** and **git push** work with that repository by default

Fetching (downloading) changes from the remote repository

```
git fetch
```

`git fetch` updates the local mirror of the remote repository:

- it downloads the new commits from the remote repository
- it updates the references `remote/remote_name/*` to match their counterpart in the remote repository.

Example: the branch `remote/origin/master` in the local repository is updated to match the new position of the branch `master` in the remote repository

Merging remote changes into the current local branch

Changes in the remote repository can be merged explicitly into the local branch by running `git merge`

```
$ git status
# On branch master
$ git fetch
...
$ git merge origin/master
```

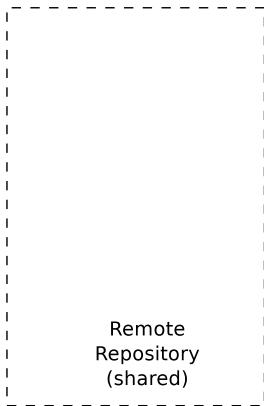
In practice, it is more convenient to use `git pull`, which is a shortcut for `git fetch + git merge`

```
git pull
```

```
$ git pull
```


Remote example

```
git init --bare --shared
```



Remote
Repository
(shared)

Remote example

`git init`

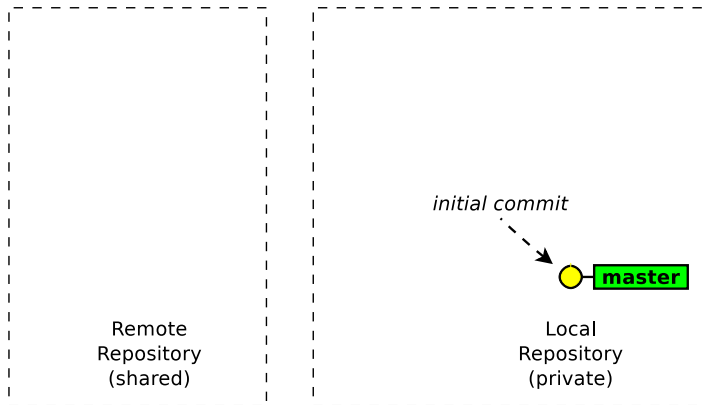


Remote
Repository
(shared)

Local
Repository
(private)

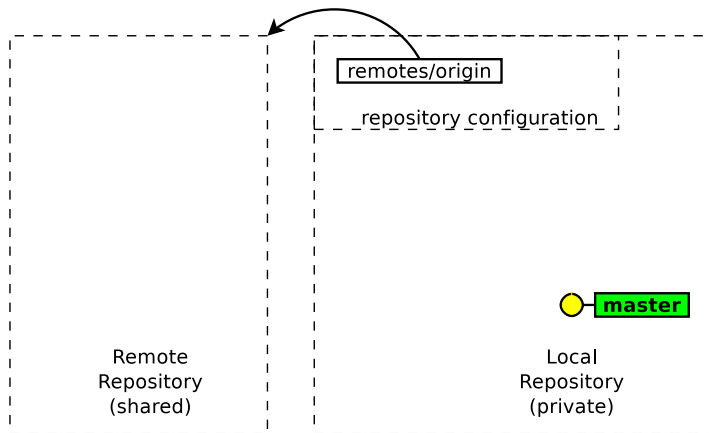
Remote example

`git commit`



Remote example

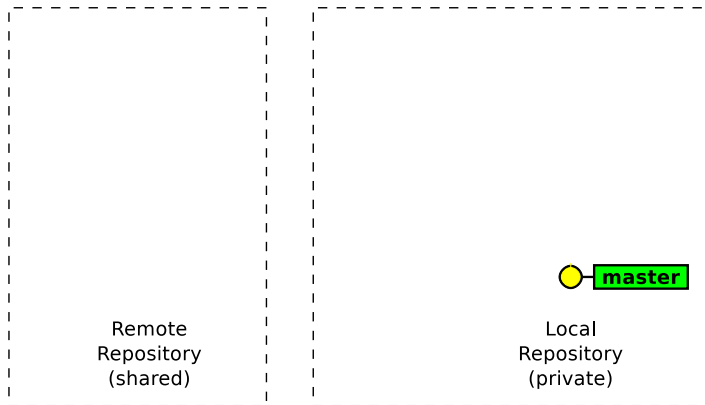
```
git remote add origin shared_url
```



Remote example

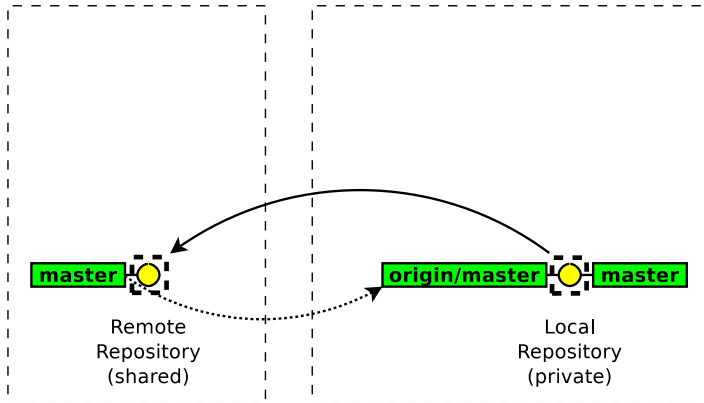
git push

-> nothing to be pushed !!



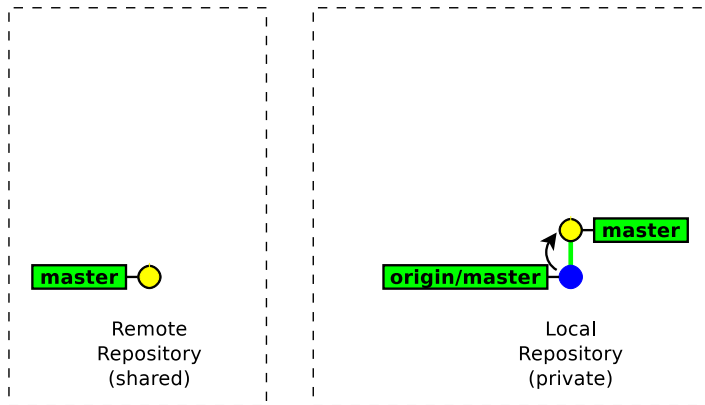
Remote example

```
git push -u origin master
```



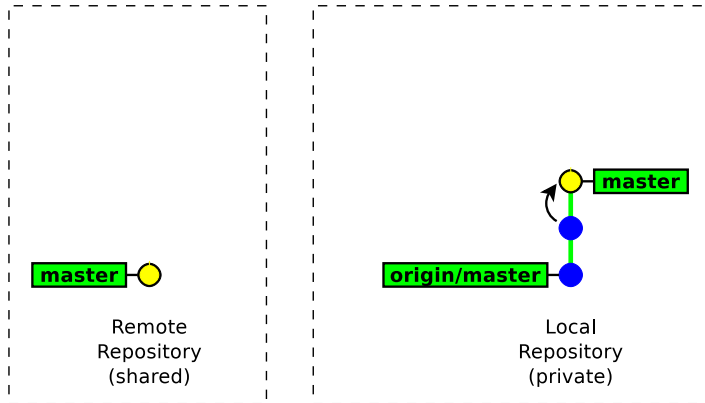
Remote example

`git commit`



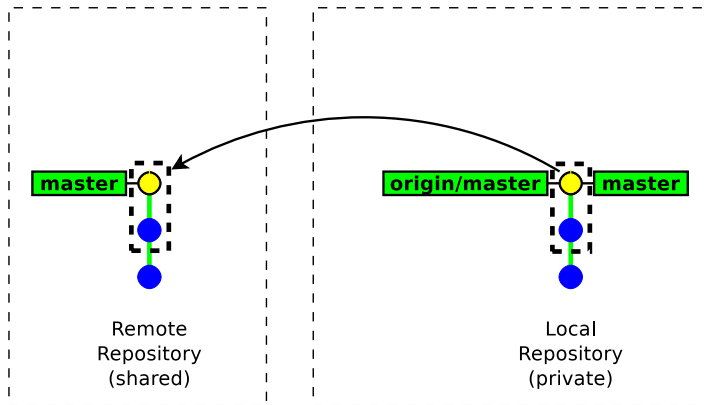
Remote example

`git commit`



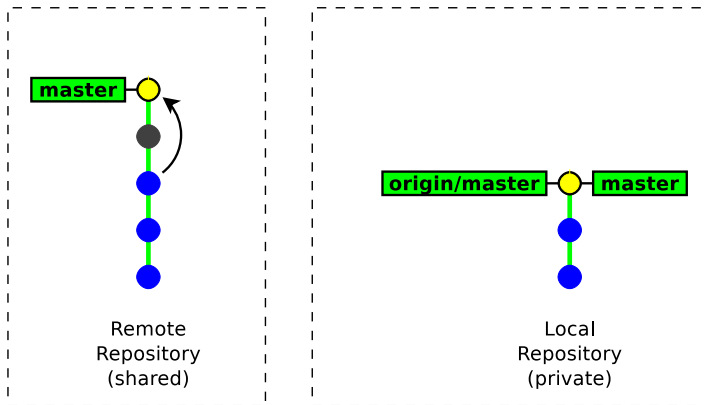
Remote example

`git push`



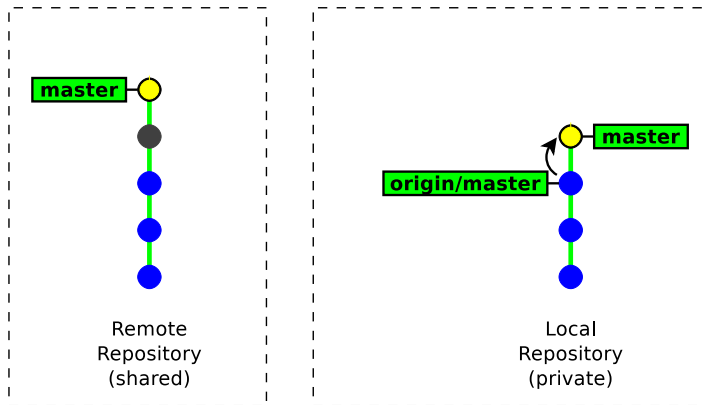
Remote example

*another developer
pushes his two commits*



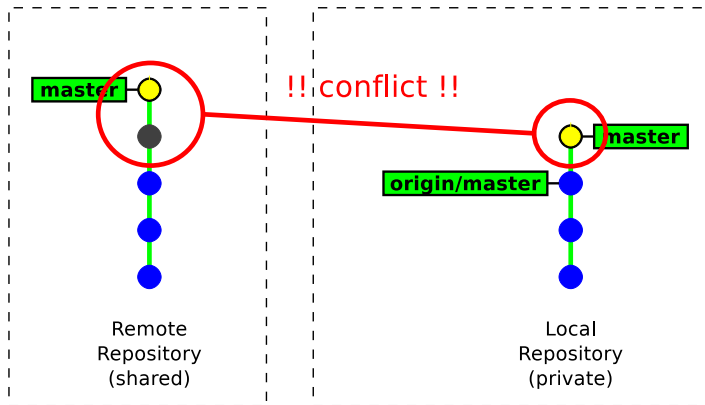
Remote example

`git commit`



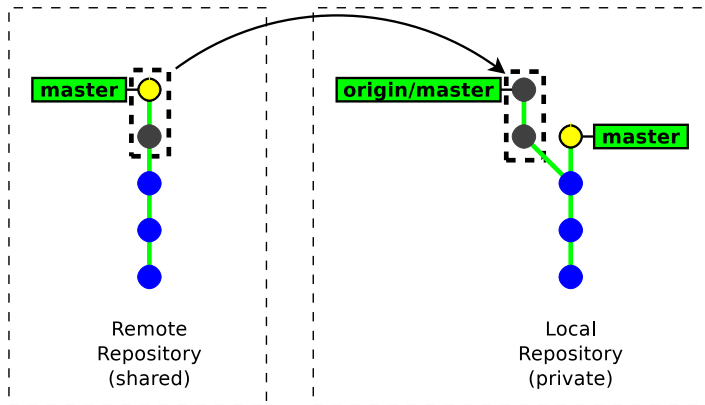
Remote example

git push



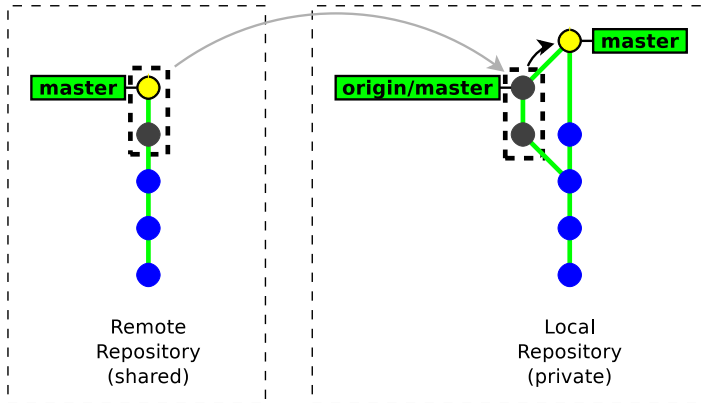
Remote example

git fetch



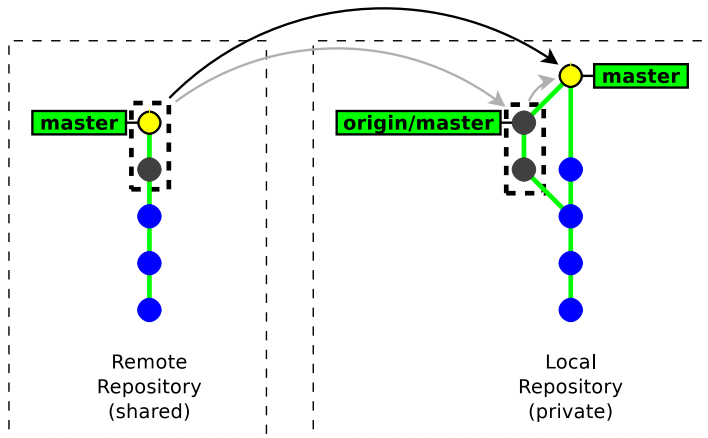
Remote example

`git merge origin/master`



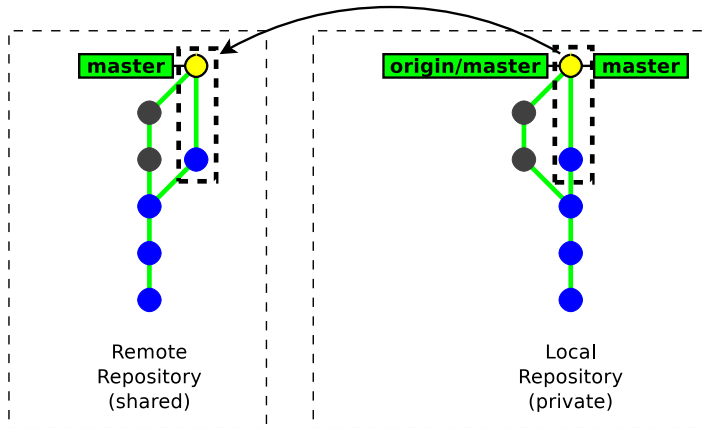
Remote example

git pull



Remote example

git push



Importing a new remote branch

```
git checkout branch_name
```

If the *branch_name* does not exist locally, then GIT looks for it in the remote repositories. If it finds it, then it creates the local branch and configures it to track the remote branch.

```
$ git branch --all
```

```
* master
```

```
remotes/origin/master
```

```
remotes/origin/new-fancy-feature
```

```
$ git checkout new-fancy-feature
```

```
Branch new-fancy-feature set up to track remote branch new-fancy-feature from origin.
```

```
Switched to a new branch 'new-fancy-feature'
```

```
$ git branch --all
```

```
master
```

```
* new-fancy-feature
```

```
remotes/origin/master
```

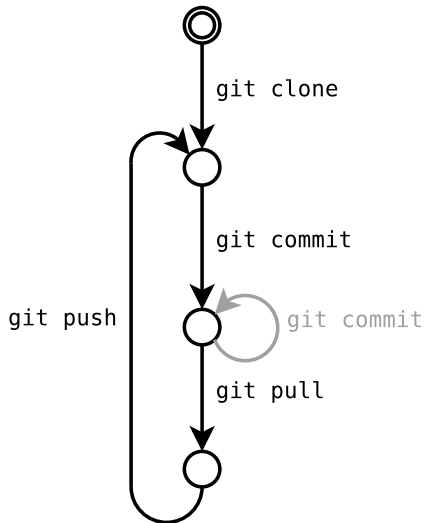
```
remotes/origin/new-fancy-feature
```

Cloning a repository

```
git clone url [ directory ]
```

- `git clone` makes a local copy of a remote repository and configures it as its origin remote repository.
- `git clone` is a shortcut for the following sequence:
 1. `git init directory`
 2. `cd directory`
 3. `git remote add origin url`
 4. `git fetch`
 5. `git checkout master`
- In practice you will rarely use `git init`, `git remote` and `git fetch` directly, but rather use higher-level commands: `git clone` and `git pull`.

Typical Workflow



Exercises

0. *(remember to visualise your operations with “gitk --all” → hit F5)*
1. clone the following repository <https://allgo.inria.fr/git/hello>
2. use `gitk --all` (to display remote branches too)
3. make some commits and synchronise (`pull/push`) with the origin repository
4. do it again so as to experience and resolve a conflict
5. use `git fetch` to review remote commits **before** merging them
6. create a new branch, make a commit and publish it to the shared repository
7. check out a branch created by another participant

Part 6.

Hosting your code

- GIT forges
- The bug tracker
- Continuous Integration / Continuous Deployment

What is a forge (1/2)

- Remote repositories need to be hosted somewhere on the network
- The most popular way to use *forge*, which hosts the repository as a *project*, along with lot of features useful for software development
- The most popular forge are:
 - [Github](#) (SAAS)
 - [Gitlab](#) (SAAS, also available in open-source, eg: <https://gitlab.istic.univ-rennes1.fr/>)

What is a forge (2/2)

- For each hosted *project*, the forge typically provides:
 - a GIT repository
 - a bug tracker
 - *issues* (bug reports, todo items, roadmap)
 - *pull requests/merge requests* (code submission, code review)
 - continuous integration & deployment (automated tasks)
 - package registries (to publish binary artifacts, eg: maven packages, docker images)
 - a wiki and/or a static web site (*pages*)
 - a WEB editor
 - an API for plugging external tools (eg. analytics, security scanner, ...)
 - fine-grained access control
 - ...

Bug tracker (1/2)

Work items are referred as *issues*:

- each issue has a unique id, prefixed with # (eg: #1, #2)
- it contains a text description of the problem and a discussion
- issues may be grouped by tags and/or by milestone
- issues may be assigned to a developer or team
- complex issues may be subdivided into smaller issues
- repository integration: issues are automatically closed when the text “Fix #*id*” or “Close #*id*” is present in:
 - the message of a commit merged into the main branch
 - the description of a pull request merged into the main branch

Bug tracker (2/2)

Code submissions are referred as *pull request* (in Github) or *merge request* (in Gitlab):

- each request has a unique id, prefixed with # (in Github) or ! (in Gitlab)
- it refers to a git branch (possibly hosted in another project) to be merged into the current project
- it contains a text description of the implementation and a discussion
- it facilitates code review before accepting a submission, the UI displays a diff of the submitted code (global diff, per-commit diff, per-update diff when the origin branch is updated)
- closing a request may be done with merging (accepted) or without merging (rejected)
- a request may refer to one or more issues, multiple pull requests may refer to the same issue (in case multiple implementation were submitted, possibly from different people)

Continuous Integration / Continuous Deployment (1/2)

Automatic tasks may be scripted in `.gitlab-ci.yml` (in Gitlab) or in `.github/workflows/` (in Github)

- it may be used for:
 - Continuous Integration (eg: build the software, run the testsuites), so as to detect bugs as soon as possible
 - Continuous Deployment (eg: publish artifacts, deploy the service, update the website/generated doc)
- CI/CD jobs are run on a *runner* outside the forge, runners may be dedicated to a project/group (private runner) or available to any project (shared runner)
- the same job may be run on multiple runners (eg: to support multiple target platforms)
- when a job fails, the user triggering it is notified by e-mail

Continuous Integration / Continuous Deployment (2/2)

security consideration

- be very cautious when the same project is configured to:
 - run jobs on merge requests submitted by external contributors
 - perform continuous deployment
- a malicious user may execute arbitrary code in your runner and find a path to attack your production server / infect your published artifacts (eg: by poisoning caches, by rooting the runner, ...)
- you must ensure that proper isolation is implemented.
If unsure, it is wiser to implement continuous deployment using a separate project and separate runner.
- example horror story at github:

<https://adnanthekhan.com/2023/12/20/one-supply-chain-attack-to-rule-them-all/>

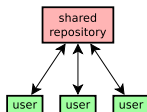
Part 7.

Working with third-party contributors

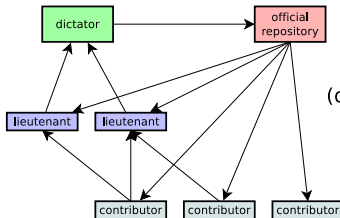
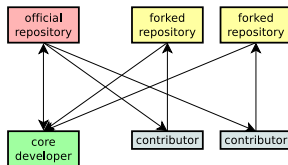
- Common workflows
- Generating & applying patches
- Merging from third-party repositories

Common workflows

Centralised



Decentralised



Hierarchical (dictator-lieutenants)

more about workflows at: <https://www.atlassian.com/git/workflows>

About 3rd party contributions

Third-party contributors¹⁶ can submit their contributions by:

- sending patches (the traditional way)
- publishing their own (unofficial) repository and asking an official developer to merge from this repository (*pull request* or *merge request*)

¹⁶developers who are not allowed to **push** to the official repository

Explicit pull/push

push/pull can work on any arbitrary repository identified by its url

```
git push url ref [ref...]
```

```
git push url local_ref:remote_ref ...    (push as a different name)
```

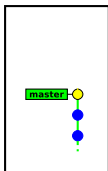
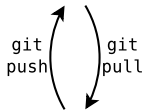
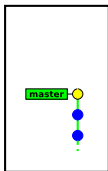
→ push the local *ref* (a branch or a tag) to repository *url*

```
git pull url ref [ref...]
```

→ merge the remote *ref* (a branch or a tag) from repository *url* into the current local branch

Decentralised workflow

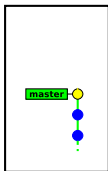
Official repository



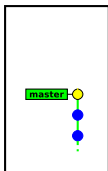
Developer

Decentralised workflow

Official repository



`git clone https://official/repo`



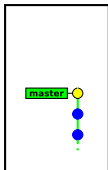
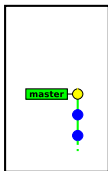
Developer



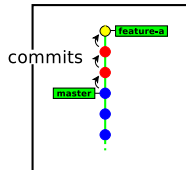
External Contributor

Decentralised workflow

Official repository



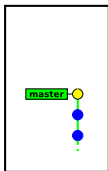
Developer



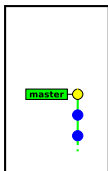
External Contributor

Decentralised workflow

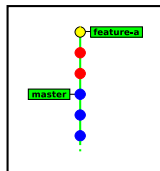
Official repository



git push



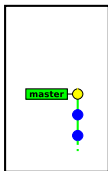
Developer



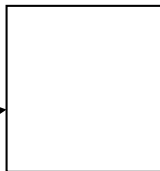
External
Contributor

Decentralised workflow

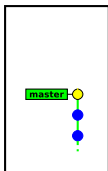
Official repository



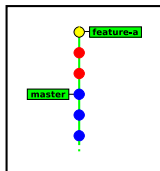
Unofficial repository



create a new
remote repository



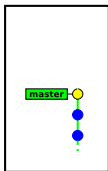
Developer



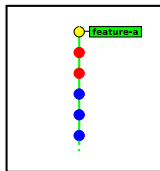
External
Contributor

Decentralised workflow

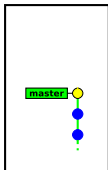
Official repository



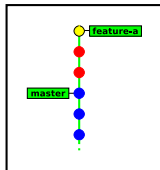
Unofficial repository



`git push https://my.unofficial/repo feature-a`



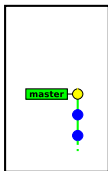
Developer



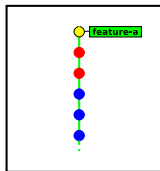
External Contributor

Decentralised workflow

Official repository

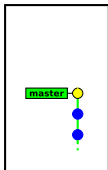


Unofficial repository

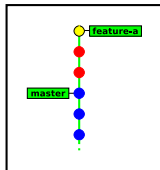


```
git remote add my-repo https://my.unofficial/repo
```

```
git push my-repo feature-a
```



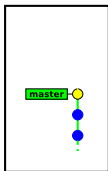
Developer



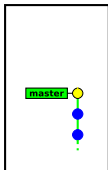
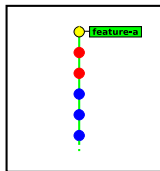
External Contributor

Decentralised workflow

Official repository

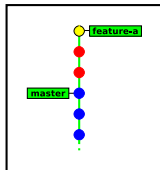
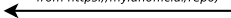


Unofficial repository



Developer

merge request
(please merge branch feature-a
from <https://my.unofficial/repo>)



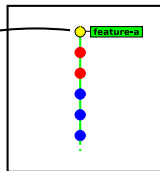
External
Contributor

Decentralised workflow

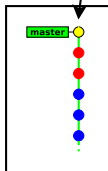
Official repository



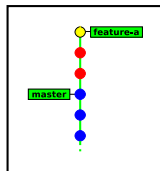
Unofficial repository



`git pull https://my.unofficial/repo feature-a`



Developer



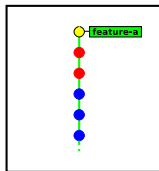
External
Contributor

Decentralised workflow

Official repository



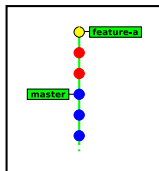
Unofficial repository



git push



Developer



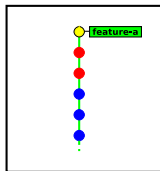
External Contributor

Decentralised workflow

Official repository



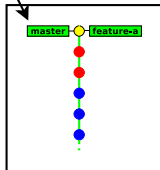
Unofficial repository



git checkout master
git pull



Developer

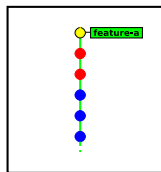
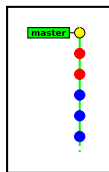


External
Contributor

Decentralised workflow

Official repository

Unofficial repository

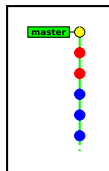


push
↑

pull
↙

pull
↘

push
↑



Developer

External
Contributor

Reviewing a remote branch

`git pull` merges **immediately** the remote branch into the current local branch.

In practice you may prefer to review it before merging.

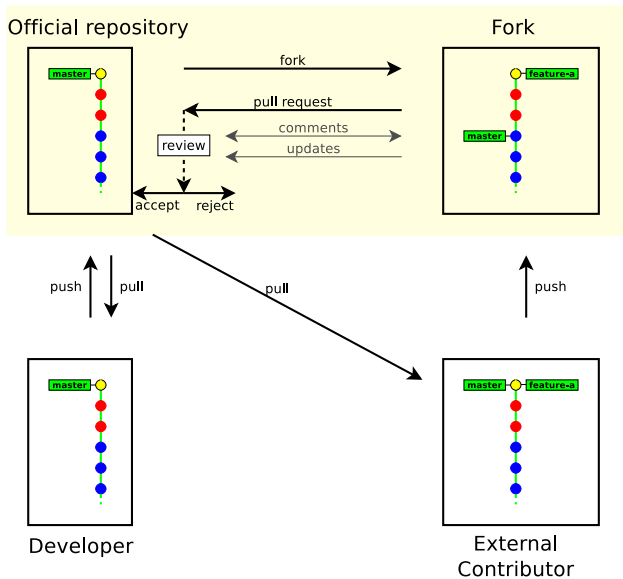
```
git fetch url branch
```

→ fetch the branch *branch* from the repository *url* and store it temporarily¹⁷ as `FETCH_HEAD`

```
$ git fetch git://git.raoul-duke.org/helloworld.git master
From git://git.raoul-duke.org/helloworld.git
* branch      master    -> FETCH_HEAD
$ gitk FETCH_HEAD
...review the commits ...
$ git merge FETCH_HEAD
```

¹⁷the `FETCH_HEAD` ref remains valid until the next time `git fetch` is run

Decentralised workflow (GIT-centric forges)



Generating patches

- `git diff`

The basic (legacy) way: use `git diff`

- `git format-patch`

The modern way: `git format-patch` converts you history (commits) into a series of patches (on file per commit) and it **records the metadata (author name, commit message)** ¹⁸

¹⁸**Note:** `git format-patch` does not preserve merge history & conflicts resolution. You should only use it when your history is linear.

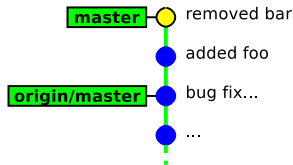
Generating patches

```
git format-patch rev_origin[..rev_final ]
```

`git format-patch` generates patches from revision *rev_origin* to *rev_final* (or to the current version if not given)

Example:

```
$ git format-patch origin/master  
0001-added-foo.patch  
0002-removed-bar.patch
```



Applying patches

```
git am file1 [ file2 ...]
```

- `git am`¹⁹ applies a series of patches generated by `git format-patch` into the local repository (each patch produces one commit)
- the authorship of the submitter is preserved²⁰

```
$ git am 0001-added-foo.patch 0002-removed-bar.patch
```

```
Applying: added foo
```

```
Applying: removed bar
```

¹⁹`am` originally stands for “apply mailbox”

²⁰actually GIT distinguishes between the **author** and the **committer** of a revision (usually they refer to the same person, but not when running `git am`)

Exercises

1. look up the *helloforge* project on the istic forge at:
<https://gitlab.istic.univ-rennes1.fr/abaire/helloforge>
2. clone the repository (click on the *Clone* button to get the repository url, prefer using the ssh url rather than https)
3. create one or more commit, then try to push your master branch (note: this should fail because the master branch is protected)
4. push your branch under a different name, eg:

```
$ git push origin master:my_new_branch_name
```

5. create a merge request for your branch (to merge it into origin/master). You can do that either in the *merge requests* page on the forge or by following the url displayed by the *git push* command.
6. once your merge request is created, browse its details (discussion, changes, ...), then approve it to have it merged into the master branch.
7. run *git pull* and check the results in *gitk*
8. repeat steps 3 to 7, but by publishing your branche in a forked repository (click on *Fork* on the project page to create your fork)

Part 8.

Extras

- Some advices
- Common traps
- Documentation
- Next tutorial

Some advices (1/2)

- commit as often as you can (keep independent changes in separate commits)
- run `git diff` before preparing a commit
- in commit messages, describe the rationale behind of your changes (it is often more important than the change itself)
- do not forget to run `git push`
- use a `.gitignore` file to ignore generated files (*.o, *.a, ...)

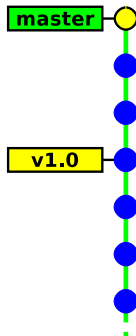
Common traps (1/2)

- `git diff` without arguments shows the difference with the index → run `git diff HEAD` to show the differences with the last commit
- `git reset` reverts the index, but keeps the working copy unchanged
→ do `git reset --hard` if you need to revert the working copy too

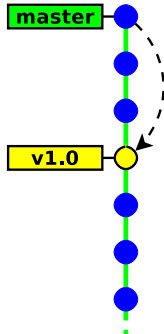
Common traps (2/2)

- GIT is not forgiving, do not ignore its warnings and do not use `--force` unless you have a clear idea of what you are doing
- GIT's history is not immutable
- `git checkout` on an arbitrary commit or a tag (anything that is not a branch) puts you in “detached HEAD” state. You can commit, but your history can be lost if you don't create any branch (or tag) to reference them.

Detached head state

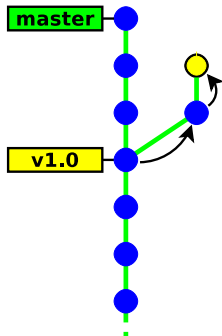


Detached head state



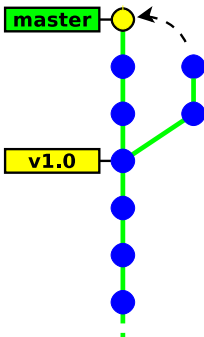
`git checkout v1.0`

Detached head state



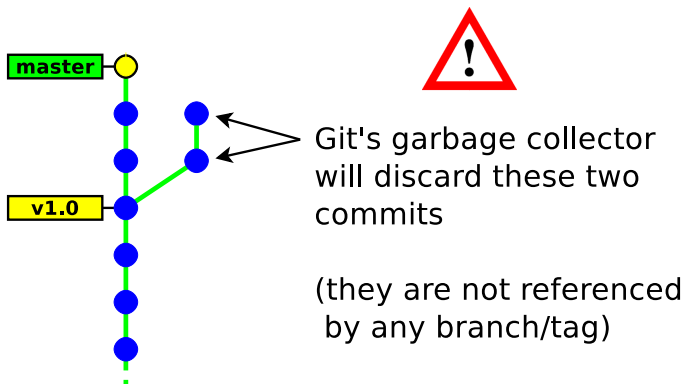
```
git commit  
git commit
```

Detached head state



git checkout master

Detached head state



Other useful utility commands

- `git gc` → garbage collector (run it when the `/.git/` directory takes too much space)
- `git stash` → save/restore the state of the working copy and index (useful when in need to commit an urgent fix)
- `git clean` → clean the working tree (⚠ you must ensure that all your code is committed)
- `git bisect` → locating which commit introduced a bug
- `git cherry-pick` → merging a single commit
- `git revert` → cancelling a previous commit

Further documentation

- `man git cmd` (tough & exhaustive)
- `man gitglossary`
- The Git book

<http://git-scm.com/book>

- The Git community book

<http://www.scribd.com/doc/7502572/The-Git-Community-Book>

- Github learning materials

<http://learn.github.com/>

- Atlassian learning materials

<https://www.atlassian.com/git/tutorial>

<https://www.atlassian.com/git/workflows>

- Tech Talk: Linus Torvalds on git (May 2007)

<https://www.youtube.com/watch?v=4XpnKHJAok8>